

Name: Shriya Vasundhara Singh

SRN: PES1UG24AM269

LAB 3 SUBMISSION

Group Travel Expense Simplification Engine

Project Title	Group Travel Expense Simplification Engine
Primary Domain	Retail E-Commerce and Finance
Target Actors	Group Traveler, Trip Admin
Selected Architecture	Microservices Architecture
Deliverable 1	Architecture Selection & Justification - 8 Marks
Deliverable 2	UML Component Diagram - 7 Marks

Submission contents: Architecture justification and UML component diagram.

1. Architecture Selection & Justification - 8 Marks

Selected Architecture: Microservices Architecture

We chose **Microservices Architecture** for the Group Travel Expense Simplification Engine. The system is divided into independent services such as authentication, trip management, expense management, expense splitting/simplification, payment and settlement, and notifications. Each service exposes well-defined interfaces and can be developed, deployed and scaled independently.

Reason 1 - Independent Scalability

Expense recording and balance/split calculations can experience higher demand during active trips than functions such as notifications. Microservices allows the Expense Management Service and the Expense Split & Simplification Engine to be scaled independently instead of scaling the entire application.

Reason 2 - Modularity and Fault Isolation

Trip management, expense processing, settlement and notifications are logically separate responsibilities. Separating them into services reduces the impact of failures. For example, a temporary failure in the Notification Service should not stop a traveler from recording an expense or calculating a balance.

Security Advantage

A dedicated User & Authentication Service can authenticate users and enforce role-based authorization for Group Travelers and Trip Admins. Financial operations can be isolated in the Payment & Settlement Service and exposed only through controlled APIs, limiting direct access to sensitive functions.

Performance Benefit

Frequently used operations can be handled by independently scalable services. If expense recording becomes busy, the Expense Management Service can be scaled without unnecessarily scaling authentication, notifications or other services. This improves resource utilization and responsiveness under uneven workloads.

Conclusion

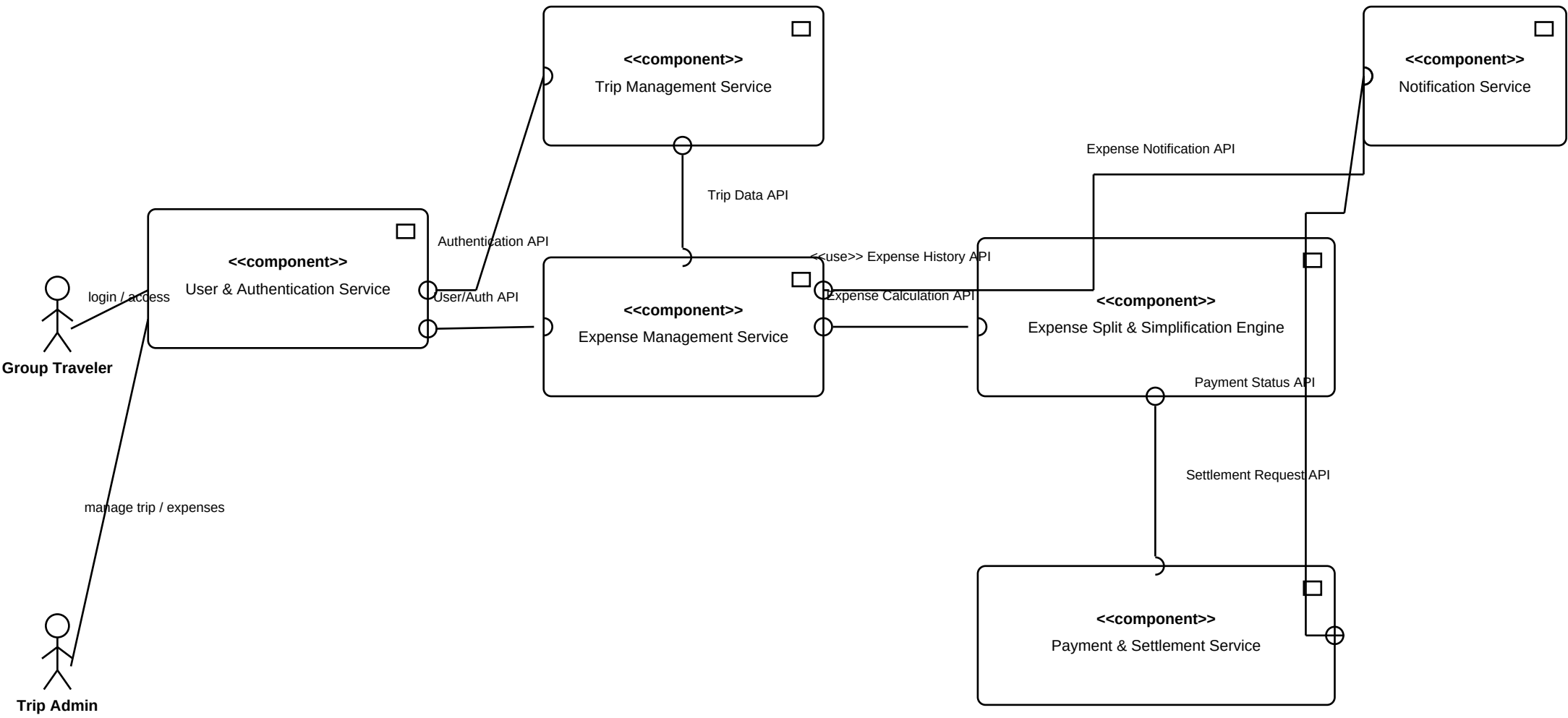
Microservices is appropriate because the project naturally consists of independent functional areas and includes finance-related operations. Its independent deployment, scalability and fault-isolation characteristics provide a strong fit, while the main trade-offs are operational complexity, network communication and distributed data-consistency management.

Architecture Comparison

Architecture	Advantages	Limitations for this project
Layered	Clear separation of concerns; easy to understand.	Tighter coupling; harder to scale individual functions.
Microservices	Independent scaling; fault isolation; independent deployment.	Operational complexity; network latency; consistency challenges.
Client-Server	Centralized control; simple deployment; easy consistency.	Central bottleneck; single point of failure; limited fault tolerance.

UML Component Diagram - Group Travel Expense Simplification Engine

Microservices Architecture | Ball = provided interface | Socket = required interface



Legend

- Provided interface
- ⌋ Required interface
- Usage dependency

Core component: Expense Split & Simplification Engine

2. UML Component Diagram - 7 Marks

The UML diagram contains six components, the two target actors, provided and required interfaces, service interaction labels, and a usage dependency. The Expense Split & Simplification Engine is the core component responsible for calculating shares, balances and optimized settlement instructions.

Component	Primary Responsibility
User & Authentication Service	Authentication, authorization, roles and profiles
Trip Management Service	Trip creation, membership and Trip Admin management
Expense Management Service	Expense recording, payer and participant information
Expense Split & Simplification Engine	Shares, balances and minimized settlement instructions
Payment & Settlement Service	Settlement execution, payment status and transaction history
Notification Service	Expense, trip, settlement and payment notifications

Interfaces shown: Authentication API, User/Auth API, Trip Data API, Expense Calculation API, Settlement Request API, Expense Notification API, Payment Status API, and an Expense History usage dependency.